

S E C O N D E D I T I O N

The Art of PostgreSQL

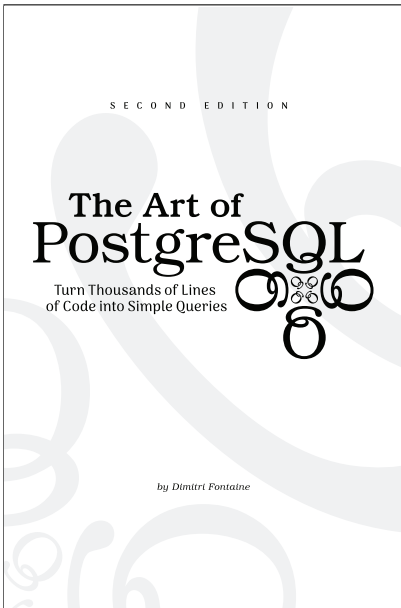
Turn Thousands of Lines
of Code into Simple Queries

by Dimitri Fontaine

This is a sample

The Art of PostgreSQL

This is a *sample* of the book THE ART OF POSTGRESQL. The sample contains the full table of contents, list of figures and index, and the introduction part of the book. The full book includes:



- 10 parts and 51 chapters
- 6 interviews
- 468 pages, 334 SQL queries
- 15 datasets from the Real World™
- 66 tables in 16 schemas
- 4 sample applications
- code written in Python and Go
- Practical examples of data normalization

Contents

I	Preface	xi
	About...	xiii
1	About the Book	xiii
2	About the Author	xiv
3	Acknowledgements	xiv
4	About the organisation of the book	xv
II	Introduction	1
1	Structured Query Language	2
1.1	Some of the Code is Written in SQL	3
1.2	A First Use Case	4
1.3	Loading the Data Set	4
1.4	Application Code and SQL	5
1.5	A Word about SQL Injection	9
1.6	PostgreSQL protocol: server-side prepared statements	10
1.7	Back to Discovering SQL	13
1.8	Computing Weekly Changes	16
2	Software Architecture	20
2.1	Why PostgreSQL?	23
2.2	The PostgreSQL Documentation	25
3	Getting Ready to read this Book	26

III	Writing Sql Queries	28
4	Business Logic	30
4.1	Every SQL query embeds some business logic	30
4.2	Business Logic Applies to Use Cases	32
4.3	Correctness	35
4.4	Efficiency	37
4.5	Stored Procedures — a Data Access API	39
4.6	Procedural Code and Stored Procedures	41
4.7	Where to Implement Business Logic?	42
5	A Small Application	43
5.1	Readme First Driven Development	43
5.2	Loading the Dataset	43
5.3	Chinook Database	45
5.4	Music Catalog	47
5.5	Albums by Artist	48
5.6	Top-N Artists by Genre	48
6	The SQL REPL — An Interactive Setup	55
6.1	Intro to psql	55
6.2	The psqlrc Setup	56
6.3	Transactions and psql Behavior	57
6.4	A Reporting Tool	58
6.5	Discovering a Schema	60
6.6	Interactive Query Editor	61
7	SQL is Code	62
7.1	SQL style guidelines	62
7.2	Comments	66
7.3	Unit Tests	67
7.4	Regression Tests	70
7.5	A Closer Look	71
8	Indexing Strategy	73
8.1	Indexing for Constraints	74
8.2	Indexing for Queries	75
8.3	Cost of Index Maintenance	75
8.4	Choosing Queries to Optimize	76

8.5	PostgreSQL Index Access Methods	76
8.6	Advanced Indexing	79
8.7	Adding Indexes	80
9	An Interview with Yohann Gabory	84
IV	SQL Toolbox	89
10	Get Some Data	91
11	Structured Query Language	93
12	Queries, DML, DDL, TCL, DCL	95
13	Select, From, Where	97
13.1	Anatomy of a Select Statement	97
13.2	Projection (output): Select	97
13.3	Data sources: From	103
13.4	Understanding Joins	104
13.5	Restrictions: Where	106
14	Order By, Limit, No Offset	109
14.1	Ordering with Order By	109
14.2	kNN Ordering and GiST indexes	111
14.3	Top-N sorts: Limit	113
14.4	No Offset, and how to implement pagination	116
15	Group By, Having, With, Union All	118
15.1	Aggregates (aka Map/Reduce): Group By	118
15.2	Aggregates Without a Group By	122
15.3	Restrict Selected Groups: Having	122
15.4	Grouping Sets	123
15.5	Common Table Expressions: With	126
15.6	Recursive Common Table Expressions	131
15.7	Distinct On	133
15.8	Result Sets Operations	135

16 Understanding Nulls	139
16.1 Three-Valued Logic	139
16.2 Not Null Constraints	141
16.3 Outer Joins Introducing Nulls	142
16.4 Using Null in Applications	143
17 Understanding Window Functions	145
17.1 Windows and Frames	146
17.2 Partitioning into Different Frames	147
17.3 Available Window Functions	149
17.4 When to Use Window Functions	151
18 Understanding Relations and Joins	152
18.1 Relations	152
18.2 SQL Join Types	154
19 An Interview with Markus Winand	157
 V Data Types	 161
20 Serialization and Deserialization	163
21 Some Relational Theory	165
21.1 Attribute Values, Data Domains and Data Types	166
21.2 Consistency and Data Type Behavior	167
22 PostgreSQL Data Types	171
22.1 Boolean	172
22.2 Character and Text	174
22.3 Server Encoding and Client Encoding	178
22.4 Numbers	181
22.5 Floating Point Numbers	183
22.6 Sequences and the Serial Pseudo Data Type	183
22.7 Universally Unique Identifier: UUID	184
22.8 Bytea and Bitstring	185
22.9 Date/Time and Time Zones	186
22.10 Time Intervals	190
22.11 Date/Time Processing and Querying	192

22.12	Network Address Types	196
22.13	Ranges	199
23	Denormalized Data Types	203
23.1	Arrays	203
23.2	Composite Types	209
23.3	XML	210
23.4	JSON	211
23.5	Enum	214
24	PostgreSQL Extensions	216
25	An interview with Grégoire Hubert	218
VI	Data Modeling	221
26	Object Relational Mapping	223
27	Tooling for Database Modeling	225
27.1	How to Write a Database Model	226
27.2	Generating Random Data	229
27.3	Modeling Example	231
28	Normalization	237
28.1	Data Structures and Algorithms	237
28.2	Normal Forms	240
28.3	Database Anomalies	241
28.4	Modeling an Address Field	243
28.5	Primary Keys	245
28.6	Surrogate Keys	245
28.7	Foreign Keys Constraints	248
28.8	Not Null Constraints	249
28.9	Check Constraints and Domains	249
28.10	Exclusion Constraints	250
29	Practical Use Case: Geonames	251
29.1	Features	254
29.2	Countries	255

29.3	Administrative Zoning	259
29.4	Geolocation Data	262
29.5	Geolocation GiST Indexing	266
29.6	A Sampling of Countries	268
30	Modelization Anti-Patterns	271
30.1	Entity Attribute Values	271
30.2	Multiple Values per Column	275
30.3	UUIDs	277
31	Denormalization	279
31.1	Premature Optimization	280
31.2	Functional Dependency Trade-Offs	280
31.3	Denormalization with PostgreSQL	281
31.4	Materialized Views	282
31.5	History Tables and Audit Trails	284
31.6	Validity Period as a Range	286
31.7	Pre-Computed Values	287
31.8	Enumerated Types	287
31.9	Multiple Values per Attribute	288
31.10	The Sparse Matrix Model	288
31.11	Partitioning	288
31.12	Other Denormalization Tools	289
31.13	Denormalize with Care	290
32	Not Only SQL	291
32.1	Schemaless Design in PostgreSQL	292
32.2	Durability Trade-Offs	295
32.3	Scaling Out	296
33	An interview with Álvaro Hernández Tortosa	298
VII	Data Manipulation and Concurrency Control	303
34	Another Small Application	305
35	Insert, Update, Delete	309
35.1	Insert Into	309

35.2	Insert Into ... Select	310
35.3	Update	312
35.4	Inserting Some Tweets	315
35.5	Delete	317
35.6	Tuples and Rows	319
35.7	Deleting All the Rows: Truncate	319
35.8	Delete but Keep a Few Rows	320
36	Isolation and Locking	321
36.1	Transactions and Isolation	322
36.2	About SSI	324
36.3	Concurrent Updates and Isolation	324
36.4	Modeling for Concurrency	326
36.5	Putting Concurrency to the Test	328
37	Computing and Caching in SQL	332
37.1	Views	333
37.2	Materialized Views	334
38	Triggers	337
38.1	Transactional Event Driven Processing	338
38.2	Trigger and Counters Anti-Pattern	340
38.3	Fixing the Behavior	341
38.4	Event Triggers	343
39	Listen and Notify	344
39.1	PostgreSQL Notifications	344
39.2	PostgreSQL Event Publication System	345
39.3	Notifications and Cache Maintenance	347
39.4	Limitations of Listen and Notify	351
39.5	Listen and Notify Support in Drivers	352
40	Batch Update, MoMA Collection	353
40.1	Updating the Data	354
40.2	Concurrency Patterns	356
40.3	On Conflict Do Nothing	357
41	An Interview with Kris Jenkins	359

VIII PostgreSQL Extensions	363
42 What's a PostgreSQL Extension?	365
42.1 Inside PostgreSQL Extensions	367
42.2 Installing and Using PostgreSQL Extensions	368
42.3 Finding PostgreSQL Extensions	369
42.4 A Primer on Authoring PostgreSQL Extensions	370
42.5 A Short List of Noteworthy Extensions	370
43 Auditing Changes with hstore	376
43.1 Introduction to <i>hstore</i>	376
43.2 Comparing hstores	377
43.3 Auditing Changes with a Trigger	377
43.4 Testing the Audit Trigger	379
43.5 From hstore Back to a Regular Record	381
44 Last.fm Million Song Dataset	383
45 Using Trigrams For Typos	390
45.1 The <code>pg_trgm</code> PostgreSQL Extension	390
45.2 Trigrams, Similarity and Searches	391
45.3 Complete and Suggest Song Titles	395
45.4 Trigram Indexing	396
46 Denormalizing Tags with intarray	398
46.1 Advanced Tag Indexing	398
46.2 Searches	399
46.3 User-Defined Tags Made Easy	402
47 The Most Popular Pub Names	404
47.1 A Pub Names Database	405
47.2 Normalizing the Data	407
47.3 Geolocating the Nearest Pub (k-NN search)	408
47.4 Indexing kNN Search	409
48 How far is the nearest pub?	411
48.1 The <code>earthdistance</code> PostgreSQL contrib	412
48.2 Pubs and Cities	413
48.3 The Most Popular Pub Names by City	416

49	Geolocation with PostgreSQL	419
49.1	Geolocation Data Loading	419
49.2	Finding an IP Address in the Ranges	423
49.3	Geolocation Metadata	424
49.4	Emergency Pub	425
50	Counting Distinct Users with HyperLogLog	427
50.1	HyperLogLog	427
50.2	Installing postgresql-hll	428
50.3	Counting Unique Tweet Visitors	429
50.4	Lossy Unique Count with HLL	433
50.5	Getting the Visits into Unique Counts	435
50.6	Scheduling Estimates Computations	437
50.7	Combining Unique Visitors	439
51	An Interview with Craig Kerstiens	441
IX	Closing Thoughts	444
X	Figures and Index	446

Part I
Preface

As a developer, *The Art of PostgreSQL* is the book you need to read in order to get to the next level of proficiency.

After all, a developer's job encompasses more than just writing code. Our job is to produce results, and for that we have many tools at our disposal. SQL is one of them, and this book teaches you all about it.

PostgreSQL is used to manage data in a centralized fashion, and SQL is used to get exactly the result set needed from the application code. An SQL result set is generally used to fill in-memory data structures so that the application can then process the data. So, let's open this book with a quote about data structures and application code:

Data dominates. If you've chosen the right data structures and organized things well, the algorithms will almost always be self-evident. Data structures, not algorithms, are central to programming.

— Rob Pike

About...

About the Book

This book is intended for developers working on applications that use a database server. The book specifically addresses the [PostgreSQL](#) RDBMS: it actually is the world's most advanced Open Source database, just like it says in the tagline on the official website. By the end of this book you'll know why, and you'll agree!

I wanted to write this book after having worked with many customers who were making use of only a fraction of what SQL and PostgreSQL are capable of delivering. In most cases, developers I met with didn't know what's possible to achieve in SQL. As soon as they realized — or more exactly, as soon as they were shown what's possible to achieve —, replacing hundreds of lines of application code with a small and efficient SQL query, then in some cases they would nonetheless not know how to integrate a raw SQL query in their code base.

Integrating a SQL query and thinking about SQL as code means using the same advanced tooling that we use when using other programming languages: versioning, automated testing, code reviewing, and deployment. Really, this is more about the developer's workflow than the SQL code itself...

In this book, you will learn best practices that help with integrating SQL into your own workflow, and through the many examples provided, you'll see all the reasons why you might be interested in doing more in SQL. Primarily, it means writing fewer lines of code. As [Dijkstra](#) said, we should count lines of code as lines spent, so by learning how to use SQL you will be able to spend less to write the same application!

The practice is pervaded by the reassuring illusion that programs are just devices like any others, the only difference admitted being that their manufacture might require a new type of craftsmen,

viz. programmers. From there it is only a small step to measuring “programmer productivity” in terms of “number of lines of code produced per month”. This is a very costly measuring unit because it encourages the writing of insipid code, but today I am less interested in how foolish a unit it is from even a pure business point of view. My point today is that, if we wish to count lines of code, we should not regard them as “lines produced” but as “lines spent”: the current conventional wisdom is so foolish as to book that count on the wrong side of the ledger.

On the cruelty of really teaching computing science, *Edsger Wybe Dijkstra*, [EWD1036](#)

About the Author

Dimitri Fontaine is a PostgreSQL Major Contributor, and has been using and contributing to Open Source Software for the better part of the last twenty years. Dimitri is also the author of the [pgloader](#) data loading utility, with fully automated support for database migration from MySQL to PostgreSQL, or from SQLite, or MS SQL... and more.

Dimitri has taken on roles such as developer, maintainer, packager, release manager, software architect, database architect, and database administrator at different points in his career. In the same period of time, Dimitri also started several companies (which are still thriving) with a strong Open Source business model, and he has held management positions as well, including working at the executive level in large companies.

Dimitri runs a blog at <http://tapoueh.org> with in-depth articles showing advanced use cases for SQL and PostgreSQL.

Acknowledgements

First of all, I'd like to thank all the contributors to the book. I know they all had other priorities in life, yet they found enough time to contribute and help make this book as good as I could ever hope for, maybe even better!

I'd like to give special thanks to my friend *Julien Danjou* who's acted as a mentor over the course of writing of the book. His advice about every part of the process

has been of great value — maybe the one piece of advice that I most took to the heart has been “write the book you wanted to read”.

I’d also like to extend my thanks to the people interviewed for this book. In order of appearance, they are Yohann Gabory from the French book “Django Avancé”, Markus Winand from <http://use-the-index-luke.com> and <http://modern-sql.com>, Grégoire Hubert author of the PHP **POMM** project, Álvaro Hernández Tortosa who created **ToroDB**, bringing MongoDB to SQL, Kris Jenkins, functional programmer and author of the **YeSQL** library for Clojure, and Craig Kerstiens, head of Cloud at **Citus Data**.

Having insights from SQL users from many different backgrounds has been valuable in achieving one of the major goals of this book: encouraging you, valued readers, to extend your thinking to new horizons. Of course, the horizons I’m referring to include SQL.

I also want to warmly thank the PostgreSQL community. If you’ve ever joined a PostgreSQL community conference, or even asked questions on the mailing list, you know these people are both incredibly smart and extremely friendly. It’s no wonder that PostgreSQL is such a great product as it’s produced by an excellent group of well-meaning people who are highly skilled and deeply motivated to solve actual users’ problems.

Finally, thank you dear reader for having picked this book to read. I hope that you’ll have a good time as you read through the many pages, and that you’ll learn a lot along the way!

About the organisation of the book

Each part of “The Art of PostgreSQL” can be read on its own, or you can read this book from the first to the last page in the order of the parts and chapters therein. A great deal of thinking has been put into the ordering of the parts, so that reading “The Art of PostgreSQL” in a linear fashion should provide the best experience.

The skill progression throughout the book is not linear. Each time a new SQL concept is introduced, it is presented with simple enough queries, in order to make it possible to focus on the new notion. Then, more queries are introduced to answer more interesting business questions.

Complexity of the queries usually advances over the course of a given part, chap-

ter after chapter. Sometimes, when a new chapter introduces a new SQL concept, complexity is reset to very simple queries again. That's because for most people, learning a new skill set does not happen in a linear way. Having this kind of difficulty organisation also makes it easier to dive into a given chapter out-of-order.

Here's a quick breakdown of what each chapter contains:

Part 1, Preface

You're reading it now, the preface is a presentation of the book and what to expect from it.

Part 2, Introduction

The introduction of this book intends to convince application developers such as you, dear reader, that there's more to SQL than you might think. It begins with a very simple data set and simple enough queries, that we compare to their equivalent Python code. Then we expand from there with a very important trick that's not well known, and a pretty advanced variation of it.

Part 3, Writing SQL Queries

The third part of the book covers how to write a SQL query as an application developer. We answer several important questions here:

- Why use SQL rather than your usual programming language?
- How to integrate SQL in your application source code?
- How to work at the SQL prompt, the `psql` REPL?
- What's an indexing strategy and how to approach indexing?

A simple Python application is introduced as a practical example illustrating the different answers provided. In particular, this part insists on when to use SQL to implement business logic.

Part 3 concludes with an interview with Yohann Gabory, author of a French book that teaches how to write advanced web applications with Python and Django.

Part 4, SQL Toolbox

The fourth part of “The Art of PostgreSQL” introduces most of the SQL concepts that you need to master as an application developer. It begins with the

basics, because you need to build your knowledge and skill set on-top of those foundations.

Advanced SQL concepts are introduced with practical examples: every query refers to a data model that's easy to understand, and is given in the context of a “business case”, or “user story”.

This part covers SQL clauses and features such as ORDER BY and k-NN sorts, the GROUP BY and HAVING clause and GROUPING SETS, along with classic and advanced aggregates, and then window functions. This part also covers the infamous NULL, and what's a relation and a join.

Part 4 concludes with an interview with Markus Winand, author of “SQL Performance explained” and <http://use-the-index-luke.com>. Markus is a master of the SQL standard and he is a wizard on using SQL to enable fast application delivery and solid run-time performances!

Part 5, Data Types

The fifth part of this book covers the main PostgreSQL data types you can use and benefit from as an application developer. PostgreSQL is an ORDBMS: Object-Relational Database Management System. As a result, data types in PostgreSQL are not just the classics numbers, dates, and text. There's more to it, and this part covers a lot of ground.

Part 5 concludes with an interview with Grégoire Hubert, author of the [POMM](#) project, which provides developers with unlimited access to SQL and database features while proposing a high-level API over low-level drivers.

Part 6, Data Modeling

The sixth part of “The Art of PostgreSQL” covers the basics of relational data modeling, which is the most important skill you need to master as an application developer. Given a good database model, every single SQL query is easy to write, things are kept logical, and data is kept clean. With a bad design... well my guess is that you've seen what happens with a not-great data model already, and in many cases that's the root of developers' dislike for the SQL language.

This part comes late in the book for a reason: without knowledge of some of the advanced SQL facilities, it's hard to anticipate that a data model is going to be easy enough to work with, and developers then tend to apply early optimizations to

the model to try to simplify writing the code. Well, most of those *optimizations* are detrimental to our ability to benefit from SQL.

Part 6 concludes with an interview with Álvaro Hernández Tortosa, who built the [ToroDB](#) project, a MongoDB replica solution based on PostgreSQL! His take on relational database modeling when compared to NoSQL and document based technologies and APIs is the perfect conclusion of the database modeling part.

Part 7, Data Manipulation and Concurrency Control

The seventh part of this book covers DML and concurrency, the heart of any live database. DML stands for “Data Manipulation Language”: it’s the part of SQL that includes INSERT, UPDATE, and DELETE statements.

The main feature of any RDBMS is how it deals with concurrent access to a single data set, in both reading and writing. This part covers isolation and locking, computing and caching in SQL complete with cache invalidation techniques, and more.

Part 7 concludes with an interview with Kris Jenkins, a functional programmer and open-source enthusiast. He mostly works on building systems in Elm, Haskell & Clojure, improving the world one project at a time, and he’s the author of the [YeSQL](#) library.

Part 8, PostgreSQL Extensions

The eighth part of “The Art of PostgreSQL” covers a selection of very useful PostgreSQL Extensions and their impact on simplifying application development when using PostgreSQL.

We cover auditing changes with `hstore`, the `pg_trgm` extension to implement auto-suggestions and auto-correct in your application search forms, user-defined tags and how to efficiently use them in search queries, and then we use `ip4r` for implementing geolocation oriented features. Finally, `hyperloglog` is introduced to solve a classic problem with high cardinality estimates and how to combine them.

Part 8 concludes with an interview with Craig Kerstiens who heads the Cloud team at Citus Data, after having been involved in PostgreSQL support at

Heroku. Craig shares his opinion about using PostgreSQL extensions when deploying your application using a cloud-based PostgreSQL solution.

Part II

Introduction

1

Structured Query Language

SQL stands for *Structured Query Language*; the term defines a declarative programming language. As a user, we declare the result we want to obtain in terms of a data processing pipeline that is executed against a known database model and a dataset.

The database model has to be statically declared so that we know the type of every bit of data involved at the time the query is carried out. A query result set defines a relation, of a type determined or inferred when parsing the query.

When working with SQL, as a developer we relatedly work with a type system and a kind of relational algebra. We write code to retrieve and process the data we are interested in, in the specific way we need.

RDBMS and SQL are forcing developers to think in terms of data structure, and to declare both the data structure and the data set we want to obtain via our queries.

Some might then say that SQL forces us to be good developers:

I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important. Bad programmers worry about the code. Good programmers worry about data structures and their relationships.

— Linus Torvalds

Some of the Code is Written in SQL

If you're reading this book, then it's easy to guess that you are already maintaining at least one application that uses SQL and embeds some SQL queries into its code.

The SQLite project is another implementation of a SQL engine, and one might wonder if it is the [Most Widely Deployed Software Module of Any Type?](#)

SQLite is deployed in every Android device, every iPhone and iOS device, every Mac, every Windows10 machine, every Firefox, Chrome, and Safari web browser, every installation of Skype, every version of iTunes, every Dropbox client, every TurboTax and Quick-Books, PHP and Python, most television sets and set-top cable boxes, most automotive multimedia systems.

The page goes on to say that other libraries with similar reach include:

- The original zlib implementation by Jean-loup Gailly and Mark Adler,
- The original reference implementation for *libpng*,
- *Libjpeg* from the Independent JPEG Group.

I can't help but mention that *libjpeg* was developed by [Tom Lane](#), who then contributed to developing the specs of *PNG*. Tom Lane is a Major Contributor to the PostgreSQL project and has been for a long time now. Tom is simply one of the most important contributors to the project.

Anyway, SQL is very popular and it is used in most applications written today. Every developer has seen some `select ... from ... where ...` SQL query string in one form or another and knows some parts of the very basics from SQL-89.

The current SQL standard is SQL:2016 and it includes many advanced data processing techniques. If your application is already using the SQL programming language and SQL engine, then as a developer it's important to fully understand how much can be achieved in SQL, and what service is implemented by this run-time dependency in your software architecture.

Moreover, this service is stateful and hosts all your application user data. In most cases user data is managed by the Relational Database Management System that is at the heart of the application code we write, and our code means nothing if we do not have the production data set that delivers value to users.

SQL is a very powerful programming language, and it is a declarative one. It's a wonderful tool to master, and once used properly it allows one to reduce both code size and the development time for new features. This book is written so that you think of good SQL utilization as one of our greatest advantages when writing an application, coding a new business case or implementing a user story!

A First Use Case

[Intercontinental Exchange](#) provides a chart with [Daily NYSE Group Volume in NYSE Listed, 2017](#). We can fetch the *Excel* file which is actually a *CSV* file using *tab* as a separator, remove the headings and load it into a PostgreSQL table.

Loading the Data Set

Here's what the data looks like with comma-separated thousands and dollar signs, so we can't readily process the figures as numbers:

2010	1/4/2010	1,425,504,460	4,628,115	\$38,495,460,645
2010	1/5/2010	1,754,011,750	5,394,016	\$43,932,043,406
2010	1/6/2010	1,655,507,953	5,494,460	\$43,816,749,660
2010	1/7/2010	1,797,810,789	5,674,297	\$44,104,237,184

So we create an ad-hoc table definition, and once the data is loaded we then transform it into a proper SQL data type, thanks to *alter table* commands.

```

1  begin;
2
3  drop table if exists factbook;
4
5  create table factbook
6  (
7      year      int,
8      date      date,
9      shares    text,
10     trades    text,
11     dollars    text
12 );
13
14 -- datestyle of the database to ISO, MDY
15 \copy factbook from 'factbook.csv' with delimiter E'\t' null ''
16
17 alter table factbook
18     alter shares
19     type bigint
20     using replace(shares, ',', '')::bigint,
```

```

21 alter trades
22   type bigint
23   using replace(trades, ',', '')::bigint,
24
25 alter dollars
26   type bigint
27   using substring(replace(dollars, ',', '') from 2)::numeric;
28
29 commit;
30

```

We use the PostgreSQL copy functionality to stream the data from the CSV file into our table. The `\copy` variant is a *psql* specific command and initiates *client/server* streaming of the data, reading a local file and sending its content through any established PostgreSQL connection.

factbook	
year	integer
date	date
shares	bigint
trades	bigint
dollars	numeric

A relation is a set of tuples sharing the same attributes, each with a fixed *data type*. PostgreSQL's static type system ensures every row conforms to this schema — year, date, and the monetary values each enforce different constraints and operations.

Figure 1.1: The factbook relation.

The factbook relation illustrates how each column in a PostgreSQL table declares a fixed data type and each row is one tuple in the set. The columns year, date, shares, trades, and dollars each enforce different constraints and operations through PostgreSQL's static type system, ensuring every row conforms to the schema.

Application Code and SQL

Now a classic question is how to list the *factbook* entries for a given month, and because the calendar is a complex beast, we naturally pick February 2017 as our example month.

The following query lists all entries we have in the month of February 2017:

```

1 \set start '2017-02-01'
2
3 select date,
4         to_char(shares, '99G999G999G999') as shares,
5         to_char(trades, '99G999G999') as trades,
6         to_char(dollars, 'L99G999G999G999') as dollars
7 from factbook
8 where date >= date :start'
9        and date < date :start' + interval '1 month'
10 order by date;
```

We use the *psql* application to run this query, and *psql* supports the use of variables. The `\set` command sets the `'2017-02-01'` value to the variable `start`, and then we re-use the variable with the expression `:start`.

The writing `date :start` is equivalent to `date '2017-02-01'` and is called a *decorated literal* expression in PostgreSQL. This allows us to set the data type of the literal value so that the PostgreSQL query parser won't have to guess or infer it from the context.

This first SQL query of the book also uses the *interval* data type to compute the end of the month. Of course, the example targets February because the end of the month has to be computed. Adding an *interval* value of `1 month` to the first day of the month gives us the first day of the next month, and we use the *less than* (`<`) strict operator to exclude this day from our result set.

The `to_char()` function is documented in the PostgreSQL section about [Data Type Formatting Functions](#) and allows converting a number to its text representation with detailed control over the conversion. The format is composed of *template patterns*. Here we use the following patterns:

- Value with the specified number of digits
- *L*, currency symbol (uses locale)
- *G*, group separator (uses locale)

Other template patterns for numeric formatting are available — see the PostgreSQL documentation for the complete reference.

Here's the result of our query:

date	shares	trades	dollars
2017-02-01	1,161,001,502	5,217,859	\$ 44,660,060,305
2017-02-02	1,128,144,760	4,586,343	\$ 43,276,102,903
2017-02-03	1,084,735,476	4,396,485	\$ 42,801,562,275
2017-02-06	954,533,086	3,817,270	\$ 37,300,908,120

2017-02-07	1,037,660,897	4,220,252	\$ 39,754,062,721
2017-02-08	1,100,076,176	4,410,966	\$ 40,491,648,732
2017-02-09	1,081,638,761	4,462,009	\$ 40,169,585,511
2017-02-10	1,021,379,481	4,028,745	\$ 38,347,515,768
2017-02-13	1,020,482,007	3,963,509	\$ 38,745,317,913
2017-02-14	1,041,009,698	4,299,974	\$ 40,737,106,101
2017-02-15	1,120,119,333	4,424,251	\$ 43,802,653,477
2017-02-16	1,091,339,672	4,461,548	\$ 41,956,691,405
2017-02-17	1,160,693,221	4,132,233	\$ 48,862,504,551
2017-02-21	1,103,777,644	4,323,282	\$ 44,416,927,777
2017-02-22	1,064,236,648	4,169,982	\$ 41,137,731,714
2017-02-23	1,192,772,644	4,839,887	\$ 44,254,446,593
2017-02-24	1,187,320,171	4,656,770	\$ 45,229,398,830
2017-02-27	1,132,693,382	4,243,911	\$ 43,613,734,358
2017-02-28	1,455,597,403	4,789,769	\$ 57,874,495,227

(19 rows)

The dataset only has data for 19 days in February 2017. Our expectations might be to display an entry for each calendar day and fill it in with either matching data or a zero figure for days without data in our *factbook*.

Here's a typical implementation of that expectation, in Python:

```

1  #! /usr/bin/env python3
2
3  import sys
4  import psycopg2
5  import psycopg2.extras
6  from calendar import Calendar
7
8  CONNSTRING = "dbname=yesql application_name=factbook"
9
10
11 def fetch_month_data(year, month):
12     "Fetch a month of data from the database"
13     date = "%d-%02d-01" % (year, month)
14     sql = """
15     select date, shares, trades, dollars
16     from factbook
17     where date >= date %s
18         and date < date %s + interval '1 month'
19     order by date;
20     """
21     pgconn = psycopg2.connect(CONNSTRING)
22     curs = pgconn.cursor()
23     curs.execute(sql, (date, date))
24
25     res = {}
26     for (date, shares, trades, dollars) in curs.fetchall():
27         res[date] = (shares, trades, dollars)
28

```

```

29     return res
30
31
32 def list_book_for_month(year, month):
33     """List all days for given month, and for each
34     day list fact book entry.
35     """
36     data = fetch_month_data(year, month)
37
38     cal = Calendar()
39     print("%12s | %12s | %12s | %12s" %
40           ("day", "shares", "trades", "dollars"))
41     print("%12s-+-%12s-+-%12s-+-%12s" %
42           ("-" * 12, "-" * 12, "-" * 12, "-" * 12))
43
44     for day in cal.itermonthdates(year, month):
45         if day.month != month:
46             continue
47         if day in data:
48             shares, trades, dollars = data[day]
49         else:
50             shares, trades, dollars = 0, 0, 0
51
52         print("%12s | %12s | %12s | %12s" %
53               (day, shares, trades, dollars))
54
55
56 if __name__ == '__main__':
57     year = int(sys.argv[1])
58     month = int(sys.argv[2])
59
60     list_book_for_month(year, month)

```

In this implementation, we use the above SQL query to fetch our result set, and moreover to store it in a dictionary. The dict's key is the day of the month, so we can then loop over a calendar's list of days and retrieve matching data when we have it and install a default result set (here, zeroes) when we don't have anything.

Below is the output when running the program. As you can see, we opted for an output similar to the *psql* output, making it easier to compare the effort needed to reach the same result.

```
$ ./factbook-month.py 2017 2
```

day	shares	trades	dollars
2017-02-01	1161001502	5217859	44660060305
2017-02-02	1128144760	4586343	43276102903
2017-02-03	1084735476	4396485	42801562275
2017-02-04	0	0	0
2017-02-05	0	0	0
2017-02-06	954533086	3817270	37300908120

2017-02-07	1037660897	4220252	39754062721
2017-02-08	1100076176	4410966	40491648732
2017-02-09	1081638761	4462009	40169585511
2017-02-10	1021379481	4028745	38347515768
2017-02-11	0	0	0
2017-02-12	0	0	0
2017-02-13	1020482007	3963509	38745317913
2017-02-14	1041009698	4299974	40737106101
2017-02-15	1120119333	4424251	43802653477
2017-02-16	1091339672	4461548	41956691405
2017-02-17	1160693221	4132233	48862504551
2017-02-18	0	0	0
2017-02-19	0	0	0
2017-02-20	0	0	0
2017-02-21	1103777644	4323282	44416927777
2017-02-22	1064236648	4169982	41137731714
2017-02-23	1192772644	4839887	44254446593
2017-02-24	1187320171	4656770	45229398830
2017-02-25	0	0	0
2017-02-26	0	0	0
2017-02-27	1132693382	4243911	43613734358
2017-02-28	1455597403	4789769	57874495227

A Word about SQL Injection

An *SQL Injection* is a security breach, one made famous by the [Exploits of a Mom](#) xkcd comic episode in which we read about *little Bobby Tables*.

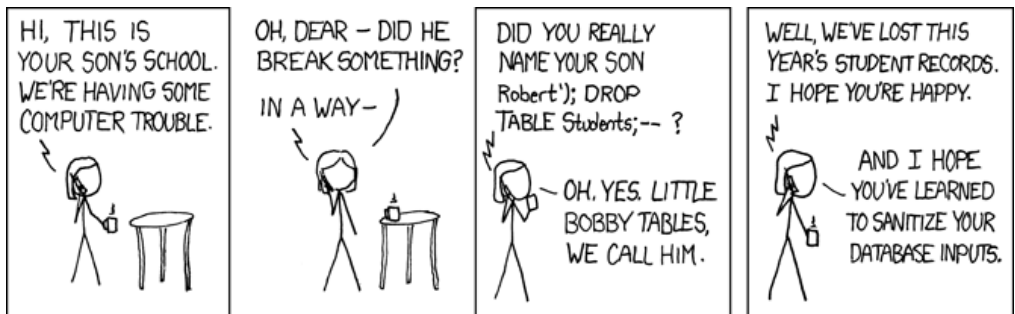


Figure 1.2: Exploits of a Mom

PostgreSQL implements a protocol level facility to send the static SQL query text separately from its dynamic arguments. An SQL injection happens when the database server is mistakenly led to consider a dynamic argument of a query as part of the query text. Sending those parts as separate entities over the protocol means that SQL injection is no longer possible.

The PostgreSQL protocol is fully documented and you can read more about *extended query* support on the [Message Flow](#) documentation page. Also relevant is the PQexecParams driver API, documented as part of the [command execution functions](#) of the libpq PostgreSQL C driver.

A lot of PostgreSQL application drivers are based on the libpq C driver, which implements the PostgreSQL protocol and is maintained alongside the main server's code. Some driver variants also exist that don't link to any C runtime, in which case the PostgreSQL protocol has been implemented in another programming language. That's the case for variants of the JDBC driver, and the pq Go driver too, among others.

It is advisable that you read the documentation of your current driver and understand how to send SQL query parameters separately from the main SQL query text; this is a reliable way to never have to worry about *SQL injection* problems ever again.

In particular, *never* build a query string by concatenating your query arguments directly into your query strings, i.e. in the application client code. Never use any library, ORM or other tooling that would do that. When building SQL query strings that way, you open your application code to serious security risk for no reason.

We were using the [psycopg](#) Python driver in our example above, which is based on libpq. The documentation of this driver addresses [passing parameters to SQL queries](#) right from the beginning.

When using Psycopg the SQL query parameters are interpolated in the SQL query string at the client level. It means you need to trust Psycopg to protect you from any attempt at SQL injection, and we could be more secure than that.

PostgreSQL protocol: server-side prepared statements

It is possible to send the query string and its arguments separately on the wire by using server-side prepared statements. This is a pretty common way to do it, mostly because PQexecParams isn't well known, though it made its debut in PostgreSQL 7.4, released November 17, 2003. To this day, a lot of PostgreSQL drivers still don't expose the PQexecParams facility, which is unfortunate.

Server-side Prepared Statements can be used in SQL thanks to the PREPARE and EXECUTE commands syntax, as in the following example:

```

1 | prepare foo as
2 |   select date, shares, trades, dollars
3 |     from factbook
4 |   where date >= $1::date
5 |         and date < $1::date + interval '1 month'
6 |   order by date;
```

And then you can execute the prepared statement with a parameter that way, still at the `psql` console:

```

1 | execute foo('2010-02-01');
```

We then get the same result as before, when using our first version of the Python program.

Now, while it's possible to use the `prepare` and `execute` SQL commands directly in your application code, it is also possible to use it directly at the PostgreSQL protocol level. This facility is named [Extended Query](#) and is well documented.

Reading the documentation about the protocol implementation, we see the following bits. First the PARSE message:

In the extended protocol, the frontend first sends a Parse message, which contains a textual query string, optionally some information about data types of parameter placeholders, and the name of a destination prepared-statement object [...]

Then, the BIND message:

Once a prepared statement exists, it can be readied for execution using a Bind message. [...] The supplied parameter set must match those needed by the prepared statement.

Finally, to receive the result set the client needs to send a third message, the EXECUTE message. The details of this part aren't relevant now though.

It is very clear from the documentation excerpts above that the query string parsed by PostgreSQL doesn't contain the parameters. The query string is sent in the BIND message. The query parameters are sent in the EXECUTE message. When doing things that way, it is impossible to have SQL injections.

Remember: SQL injection happens when the SQL parser is fooled into believing that a parameter string is in fact a SQL query, and then the SQL engine goes

on and executes that SQL statement. When the SQL query string lives in your application code, and the user-supplied parameters are sent separately on the network, there's no way that the SQL parsing engine might get confused.

The following example uses the [asyncpg](#) PostgreSQL driver. It's open source and the sources are available at the [MagicStack/asyncpg](#) repository, where you can browse the code and see that the driver implements the PostgreSQL protocol itself, and uses server-side prepared statements.

This example is now safe from SQL injection by design, because the server-side prepared statement protocol sends the query string and its arguments in separate protocol messages:

```

1  import sys
2  import asyncio
3  import asyncpg
4  import datetime
5  from calendar import Calendar
6
7  CONNSTRING = "postgresql://appdev@localhost/appdev?application_name=factbook"
8
9
10 async def fetch_month_data(year, month):
11     "Fetch a month of data from the database"
12     date = datetime.date(year, month, 1)
13     sql = """
14     select date, shares, trades, dollars
15     from factbook
16     where date >= $1::date
17         and date < $1::date + interval '1 month'
18     order by date;
19     """
20     pgconn = await asyncpg.connect(CONNSTRING)
21     stmt = await pgconn.prepare(sql)
22
23     res = {}
24     for (date, shares, trades, dollars) in await stmt.fetch(date):
25         res[date] = (shares, trades, dollars)
26
27     await pgconn.close()
28
29     return res

```

Then, the Python function call needs to be adjusted to take into account the coroutine usage we're now making via `asyncio`. The function `list_book_for_month` now begins with the following lines:

```

1  def list_book_for_month(year, month):
2      """List all days for given month, and for each

```

```

3      day list fact book entry.
4      """
5      data = asyncio.run(fetch_month_data(year, month))

```

The rest of it is as before.

Back to Discovering SQL

Now of course it's possible to implement the same expectations with a single SQL query, without any application code being *spent* on solving the problem:

```

1  select cast(calendar.entry as date) as date,
2         coalesce(shares, 0) as shares,
3         coalesce(trades, 0) as trades,
4         to_char(
5             coalesce(dollars, 0),
6             'L99G999G999G999'
7         ) as dollars
8  from /*
9         * Generate the target month's calendar then LEFT JOIN
10        * each day against the factbook dataset, so as to have
11        * every day in the result set, whether or not we have a
12        * book entry for the day.
13        */
14  generate_series(date : 'start',
15                 date : 'start' + interval '1 month'
16                      - interval '1 day',
17                 interval '1 day'
18              )
19  as calendar(entry)
20  left join factbook
21      on factbook.date = calendar.entry
22  order by date;

```

In this query, we use several basic SQL and PostgreSQL techniques that you might be discovering for the first time:

- SQL accepts comments written either in the `--` comment style, running from the opening to the end of the line, or C-style with a `/*` comment `*/` style.

As with any programming language, comments are best used to note our intentions, which otherwise might be tricky to reverse engineer from the code alone.

- `generate_series()` is a PostgreSQL [set returning function](#), for which the documentation reads:

Generate a series of values, from start to stop with a step size of step

As PostgreSQL knows its calendar, it's easy to generate all days from any given month with the first day of the month as a single parameter in the query.

- *generate_series()* is inclusive much like the *BETWEEN* operator, so we exclude the first day of the next month with the expression - *interval '1 day'*.
- The *cast(calendar.entry as date)* expression transforms the generated *calendar.entry*, which is the result of the *generate_series()* function call into the *date* data type.

We need to *cast* here because the *generate_series()* function returns a set of *timestamp* entries and we don't care about the time parts of it.

- The *left join* in between our generated *calendar* table and the *factbook* table will keep every calendar row and associate a *factbook* row with it only when the *date* columns of both the tables have the same value.

When the *calendar.date* is not found in *factbook*, the *factbook* columns (*year*, *date*, *shares*, *trades*, and *dollars*) are filled in with *NULL* values instead.

- **COALESCE** returns the first of its arguments that is not null.

So the expression *coalesce(shares, 0) as shares* is either how many shares we found in the *factbook* table for this *calendar.date* row, or 0 when we found no entry for the *calendar.date* and the *left join* kept our result set row and filled in the *factbook* columns with *NULL* values.

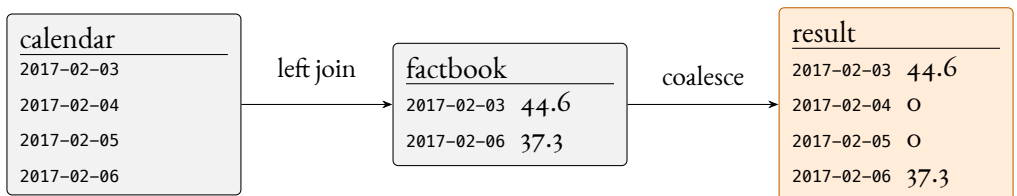


Figure 1.3: Left join between calendar and factbook.

A **LEFT JOIN** between *calendar* and *factbook* keeps every day from the *calendar*. Missing values become *NULL* for the orange result rows, then 0 via *coalesce()*.

Finally, here's the result of running this query:

date	shares	trades	dollars
2017-02-01	1161001502	5217859	\$ 44,660,060,305
2017-02-02	1128144760	4586343	\$ 43,276,102,903
2017-02-03	1084735476	4396485	\$ 42,801,562,275
2017-02-04	0	0	\$ 0
2017-02-05	0	0	\$ 0
2017-02-06	954533086	3817270	\$ 37,300,908,120
2017-02-07	1037660897	4220252	\$ 39,754,062,721
2017-02-08	1100076176	4410966	\$ 40,491,648,732
2017-02-09	1081638761	4462009	\$ 40,169,585,511
2017-02-10	1021379481	4028745	\$ 38,347,515,768
2017-02-11	0	0	\$ 0
2017-02-12	0	0	\$ 0
2017-02-13	1020482007	3963509	\$ 38,745,317,913
2017-02-14	1041009698	4299974	\$ 40,737,106,101
2017-02-15	1120119333	4424251	\$ 43,802,653,477
2017-02-16	1091339672	4461548	\$ 41,956,691,405
2017-02-17	1160693221	4132233	\$ 48,862,504,551
2017-02-18	0	0	\$ 0
2017-02-19	0	0	\$ 0
2017-02-20	0	0	\$ 0
2017-02-21	1103777644	4323282	\$ 44,416,927,777
2017-02-22	1064236648	4169982	\$ 41,137,731,714
2017-02-23	1192772644	4839887	\$ 44,254,446,593
2017-02-24	1187320171	4656770	\$ 45,229,398,830
2017-02-25	0	0	\$ 0
2017-02-26	0	0	\$ 0
2017-02-27	1132693382	4243911	\$ 43,613,734,358
2017-02-28	1455597403	4789769	\$ 57,874,495,227

(28 rows)

When ordering the book package that contains the code and the data set, you can find the SQL queries *02-intro/02-usecase/02.sql* and *02-intro/02-usecase/04.sql*, and the Python script *02-intro/02-usecase/03_factbook-month.py*, and run them against the pre-loaded database *yesql*.

Note that we replaced 60 lines of Python code with a simple enough SQL query. Down the road, that's less code to maintain and a more efficient implementation too. Here, the Python is doing a *Hash Join Nested Loop* where PostgreSQL picks a *Merge Left Join* over two ordered relations. Later in this book, we see how to get and read the PostgreSQL *execution plan* for a query.

Computing Weekly Changes

The analytics department now wants us to add a weekly difference for each day of the result. More specifically, we want to add a column with the evolution as a percentage of the *dollars* column in between the day of the value and the same day of the previous week.

I'm taking the “week over week percentage difference” example because it's both a classic analytics need, though mostly in marketing circles maybe, and because in my experience the first reaction of a developer will rarely be to write a SQL query doing all the math.

Also, computing weeks is another area in which the calendar we have isn't very helpful, but for PostgreSQL taking care of the task is as easy as spelling the word *week*:

```

1  with computed_data as
2  (
3      select cast(date as date)    as date,
4             to_char(date, 'Dy')   as day,
5             coalesce(dollars, 0)  as dollars,
6             lag(dollars, 1)
7                 over(
8                     partition by extract('isodow' from date)
9                     order by date
10                )
11         as last_week_dollars
12  from /*
13       * Generate the month calendar, plus a week before
14       * so that we have values to compare dollars against
15       * even for the first week of the month.
16       */
17       generate_series(date : 'start' - interval '1 week',
18                      date : 'start' + interval '1 month'
19                          - interval '1 day',
20                      interval '1 day'
21                )
22  as calendar(date)
23  left join factbook using(date)
24  )
25  select date, day,
26         to_char(
27             coalesce(dollars, 0),
28             'L99G999G999G999'
29         ) as dollars,
30         case when dollars is not null
31              and dollars <> 0
32              then round( 100.0

```

```

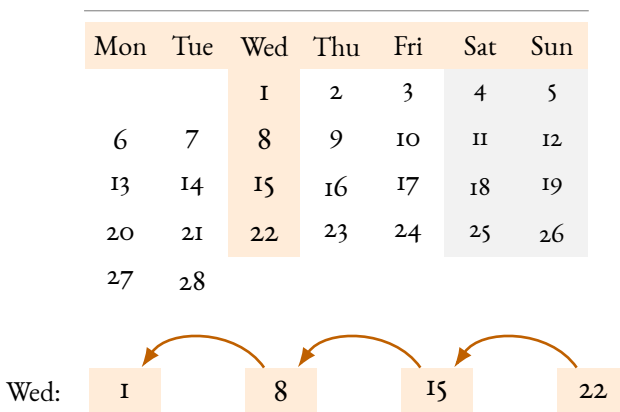
33         * (dollars - last_week_dollars)
34         / dollars
35     , 2)
36 end
37 as "WoW %"
38 from computed_data
39 where date >= date : 'start'
40 order by date;

```

To implement this case in SQL, we need *window functions* that appeared in the SQL standard in 1992 but are still often skipped in SQL classes. The last thing executed in a SQL statement are *window functions*, well after *join* operations and *where* clauses. So if we want to see a full week before the first of February, we need to extend our calendar selection a week into the past and then once again restrict the data that we issue to the caller.

That's why we use a *common table expression* — the *WITH* part of the query — to fetch the extended data set we need, including the *last_week_dollars* computed column.

The expression *extract('isodow' from date)* is a standard SQL feature that allows computing the *Day Of Week* following the *ISO* rules. Used as a *partition by* frame clause, it allows a row to be a *peer* to any other row having the same *isodow*. The *lag()* window function can then refer to the previous peer *dollars* value when ordered by date: that's the number with which we want to compare the current *dollars* value.



`lag()` over (partition by isodow order by date) — each Wednesday vs. the previous one

Figure 1.4: Window frame over February 2017 partitioned by weekday.

February 2017 is shown with partitioning by `isodow`, which groups the same weekdays together — here the Wednesdays are highlighted. Ordered by date, `lag()` reaches the previous week's value. Weekend days stand out in a pale background.

The *computed_data* result set is then used in the main part of the query as a relation we get data *from* and the computation is easier this time as we simply apply a classic difference percentage formula to the *dollars* and the *last_week_dollars* columns.

Here's the result from running this query:

date	day	dollars	WoW %
2017-02-01	Wed	\$ 44,660,060,305	-2.21
2017-02-02	Thu	\$ 43,276,102,903	1.71
2017-02-03	Fri	\$ 42,801,562,275	10.86
2017-02-04	Sat	\$ 0	∞
2017-02-05	Sun	\$ 0	∞
2017-02-06	Mon	\$ 37,300,908,120	-9.64
2017-02-07	Tue	\$ 39,754,062,721	-37.41
2017-02-08	Wed	\$ 40,491,648,732	-10.29
2017-02-09	Thu	\$ 40,169,585,511	-7.73
2017-02-10	Fri	\$ 38,347,515,768	-11.61
2017-02-11	Sat	\$ 0	∞
2017-02-12	Sun	\$ 0	∞
2017-02-13	Mon	\$ 38,745,317,913	3.73
2017-02-14	Tue	\$ 40,737,106,101	2.41
2017-02-15	Wed	\$ 43,802,653,477	7.56
2017-02-16	Thu	\$ 41,956,691,405	4.26
2017-02-17	Fri	\$ 48,862,504,551	21.52
2017-02-18	Sat	\$ 0	∞
2017-02-19	Sun	\$ 0	∞
2017-02-20	Mon	\$ 0	∞
2017-02-21	Tue	\$ 44,416,927,777	8.28
2017-02-22	Wed	\$ 41,137,731,714	-6.48
2017-02-23	Thu	\$ 44,254,446,593	5.19
2017-02-24	Fri	\$ 45,229,398,830	-8.03
2017-02-25	Sat	\$ 0	∞
2017-02-26	Sun	\$ 0	∞
2017-02-27	Mon	\$ 43,613,734,358	∞
2017-02-28	Tue	\$ 57,874,495,227	23.25

(28 rows)

The rest of the book spends some time to explain the core concepts of *common table expressions* and *window functions* and provides many other examples so that you can master PostgreSQL and issue the SQL queries that fetch exactly the result set your application needs to deal with!

We will also look at the performance and correctness characteristics of issuing more complex queries rather than issuing more queries and doing more of the processing in the application code... or in a Python script, as in the previous example.

2

Software Architecture

Our first use case in this book allowed us to compare implementing a simple feature in Python and in SQL. After all, once you know enough of SQL, lots of data related processing and presentation can be done directly within your SQL queries. The application code might then be a shell wrapper around a software architecture that is database centered.

In some simple cases, and we'll see more about that in later chapters, it is required for correctness that some processing happens in the SQL query. In many cases, having SQL do the data-related heavy lifting yields a net gain in performance characteristics too, mostly because round-trip times and latency along with memory and bandwidth resources usage depend directly on the size of the result sets.

The Art Of PostgreSQL, Volume I focuses on teaching SQL idioms, both the basics and some advanced techniques too. It also contains an approach to database modeling, normalization, and denormalization. That said, it does not address software architecture. The goal of this book is to provide you, the application developer, with new and powerful tools. Determining how and when to use them has to be done in a case by case basis.

Still, a general approach is helpful in deciding how and where to implement application features. The following concepts are important to keep in mind when learning advanced SQL:

- Relational Database Management System

PostgreSQL is an RDBMS and as such its role in your software architec-

ture is to handle concurrent access to live data that is manipulated by several applications, or several parts of an application.

Typically we will find the user-side parts of the application, a front-office and a user back-office with a different set of features depending on the user role, including some kinds of reporting (accounting, finance, analytics), and often some glue scripts here and there, crontabs or the like.

- Atomic, Consistent, Isolated, Durable

At the heart of the concurrent access semantics is the concept of a transaction. A transaction should be atomic and isolated, the latter allowing for *online backups* of the data.

Additionally, the RDBMS is tasked with maintaining a data set that is consistent with the business rules at all times. That's why database modeling and normalization tasks are so important, and why PostgreSQL supports an advanced set of *constraints*.

Durable means that whatever happens PostgreSQL guarantees that it won't lose any *committed* change. Your data is safe. Not even an OS crash is allowed to risk your data. We're left with disk corruption risks, and that's why being able to carry out *online backups* is so important.

- Data Access API and Service

Given the characteristics listed above, PostgreSQL allows one to implement a data access API. In a world of containers and micro-services, PostgreSQL is the data access service, and its API is SQL.

If it looks a lot heavier than your typical micro-service, remember that PostgreSQL implements a stateful service, on top of which you can build the other parts. Those other parts will be scalable and highly available by design, because solving those problems for *stateless* services is so much easier.

- Structured Query Language

The data access API offered by PostgreSQL is based on the SQL programming language. It's a declarative language where your job as a developer is to describe in detail the *result set* you are interested in.

PostgreSQL's job is then to find the most efficient way to access only the data needed to compute this result set, and execute the plan it comes up with.

- Extensible (JSON, XML, Arrays, Ranges)

The SQL language is statically typed: every query defines a new relation that must be fully understood by the system before executing it. That's why sometimes *cast* expressions are needed in your queries.

PostgreSQL's unique approach to implementing SQL was invented in the 80s with the stated goal of enabling extensibility. SQL operators and functions are defined in a catalog and looked up at run-time. Functions and operators in PostgreSQL support *polymorphism* and almost every part of the system can be extended.

This unique approach has allowed PostgreSQL to be capable of improving SQL; it offers a deep coverage for composite data types and documents processing right within the language, with clean semantics.

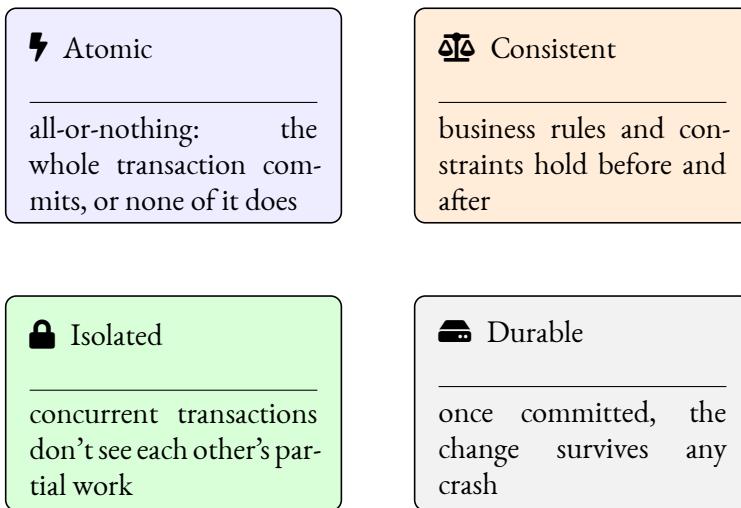


Figure 2.1: The ACID guarantees of a PostgreSQL transaction.

The four ACID guarantees a transaction provides are the heart of how PostgreSQL keeps data correct under concurrent access: **ATOMIC** means all-or-nothing, **CONSISTENT** means business rules hold before and after, **ISOLATED** means concurrent transactions do not see each other's partial work, and **DURABLE** means a committed change survives any crash.

So when designing your software architecture, think about PostgreSQL not as *storage layer*, but rather as a *concurrent data access service*. This service is capable

of handling data processing. How much of the processing you want to implement in the SQL part of your architecture depends on many factors, including team size, skill set, and operational constraints.

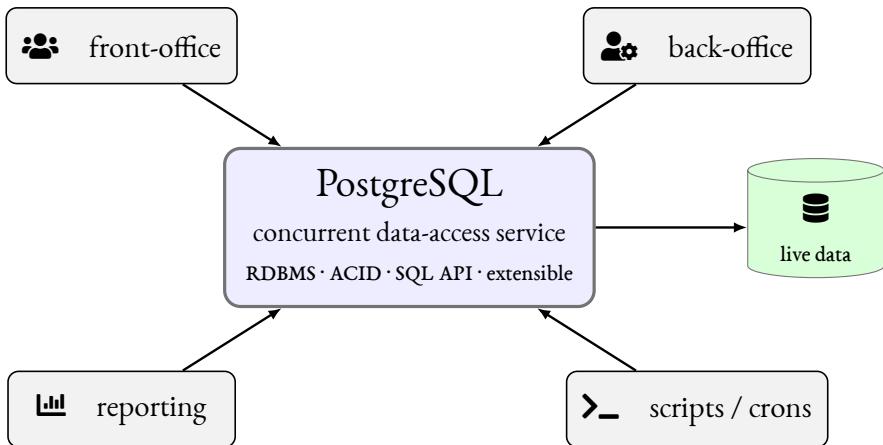


Figure 2.2: PostgreSQL as a shared data-access service.

PostgreSQL acts as a concurrent data-access service: every application component — front-office, back-office, reporting, background scripts — shares one consistent view of the live data through a single SQL API.

Why PostgreSQL?

While this book focuses on teaching SQL and how to make the best of this programming language in modern application development, it only addresses the PostgreSQL implementation of the SQL standard. That choice is down to several factors, all consequences of PostgreSQL truly being *the world's most advanced open source database*:

- PostgreSQL is open source, available under a BSD-like licence named the [PostgreSQL licence](#).
- The PostgreSQL project is done completely in the open, using public mailing lists for all discussions, contributions, and decisions, and the project goes as far as self-hosting all requirements in order to avoid being influenced by a particular company.
- While being developed and maintained in the open by volunteers, most

PostgreSQL developers today are contributing in a professional capacity, both in the interest of their employer and to solve real customer problems.

- PostgreSQL releases a new major version about once a year, following a *when it's ready* release cycle.
- The PostgreSQL design, ever since its Berkeley days under the supervision of [Michael Stonebraker](#), allows enhancing SQL in very advanced ways, as we see in the data types and indexing support parts of this book.
- The PostgreSQL documentation is one of the best reference manuals you can find, open source or not, and that's because a patch in the code is only accepted when it also includes editing the parts of the documentations that need editing.
- While new NoSQL systems are offering different trade-offs in terms of operations, guarantees, query languages and APIs, I would argue that PostgreSQL is YeSQL!

In particular, the extensibility of PostgreSQL allows this 20-year-old system to keep renewing itself. As a data point, this extensibility design makes PostgreSQL one of the best JSON processing platforms you can find.

It makes it possible to improve SQL with advanced support for new data types even from “userland code”, and to integrate processing functions and operators and their indexing support.

We'll see lots of examples of that kind of integration in the book. One of them is a query used in the [Schemaless Design in PostgreSQL](#) section where we deal with a Magic™ The Gathering set of cards imported from a JSON data set:

```

1 | select jsonb_pretty(data)
2 |   from magic.cards
3 |  where data @> '{
4 |               "type":"Enchantment",
5 |               "artist":"Jim Murray",
6 |               "colors":["White"]
7 |             }';

```

The @> operator reads *contains* and implements JSON searches, with support from a specialized GIN index if one has been created. The `jsonb_pretty()` function does what we can expect from its name, and the query returns *magic.cards* rows that match the JSON criteria for given *type*, *artist* and *colors* key, all as a pretty printed JSON document.

PostgreSQL extensibility design is what allows one to enhance SQL in that way. The query still fully respects SQL rules, there are no tricks here. It is only functions and operators, positioned where we expect them in the *where* clause for the searching and in the *select* clause for the projection that builds the output format.

The PostgreSQL Documentation

This book is not an alternative to the [PostgreSQL manual](#), which in PDF for the 9.6 server weighs in at 3376 pages if you choose the A4 format. The table of contents alone in that document runs from pages *iii* to *xxxiv*, that's 32 pages!

This book offers a very different approach than what is expected from a reference manual, and it is in no way to be considered a replacement. Bits and pieces from the PostgreSQL documentation are quoted when necessary, otherwise this book contains lots of links to the reference pages of the functions and SQL commands we utilize in our practical use cases. It's a good idea to refer to the PostgreSQL documentation and read it carefully.

After having spent some time as a developer using PostgreSQL, then as a PostgreSQL contributor and consultant, nowadays I can very easily find my way around the PostgreSQL documentation. Chapters are organized in a logical way, and everything becomes easier when you get used to browsing the reference.

Finally, the *psql* application also includes online help with `\h <sql command>`.

This book does not aim to be a substitute for the PostgreSQL documentation, and other forums and blogs might offer interesting pieces of advice and introduce some concepts with examples. At the end of the day, if you're curious about anything related to PostgreSQL: read the fine manual. No really... this one is fine.

3

Getting Ready to read this Book

Be sure to use the documentation for the version of PostgreSQL you are using, and if you're not too sure about that just query for it:

```
1 | show server_version;  
   server_version  
-----  
9.6.5  
(1 row)
```

Ideally, you will have a database server to play along with.

- If you're using MacOSX, check out [Postgres App](#) to install a PostgreSQL server and the `psql` tool.
- For Windows check <https://www.postgresql.org/download/windows/>.
- If you're mainly running Linux you know what you're doing already, right? My experience is with Debian, so have a look at <https://apt.postgresql.org> and install the most recent version of PostgreSQL on your station so that you have something to play with locally. For Red Hat packaging based systems, check out <https://yum.postgresql.org>.

In this book, we will be using `psql` a lot and we will see how to configure it in a friendly way.

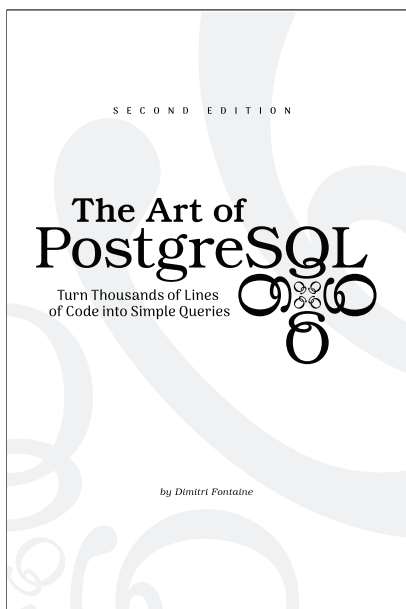
You might prefer a more visual tool such as [pgAdmin](#) or [OmniDB](#); the key here is to be able to easily edit SQL queries, run them, edit them in order to fix them,

This was the sample

The Art of PostgreSQL

This was the *sample* of the book THE ART OF POSTGRESQL. After this page the sample still includes the full list of figures and index of the book, so that you can have an idea of the material that is covered in the book.

What now? Well, buy the book!



Now that you've read the sample, and if you liked it, you might be interested into buying the whole book. It's available online at TheArtOfPostgreSQL.com in several different formats.

You'll find the BOOK EDITION, the ENTERPRISE EDITION, the FULL EDITION and of course the DEAD TREE EDITION.

Part X

Figures and Index

A decorative graphic on the right side of the page, consisting of several large, light gray, stylized swirls or loops that overlap each other, creating a sense of movement and depth.

List of Figures

1.1	The factbook relation.	5
1.2	Exploits of a Mom	9
1.3	Left join between calendar and factbook.	14
1.4	Window frame over February 2017 partitioned by weekday. . .	17
2.1	The ACID guarantees of a PostgreSQL transaction.	22
2.2	PostgreSQL as a shared data-access service.	23
5.1	The Chinook database schema.	46
8.1	Four PostgreSQL index access methods.	77
8.2	The lifecycle of a SQL query inside PostgreSQL.	80
8.3	Advanced Django	83
10.1	The Fddb schema.	92
14.1	LATERAL join semantics.	114
14.2	No Offset	116
15.1	FILTER (WHERE ...) mechanics.	121
15.2	CTE pipeline for finding Formula One champions.	129
15.3	The main channels of the Loire basin (a flat query on stream order).	132
15.4	Every reach of the Loire basin, gathered upstream with with recursive.	133
15.5	DISTINCT ON (weekday) ordered by date.	134
16.1	Three-valued logic truth tables for AND, OR and NOT.	139
17.1	GROUP BY versus window functions with PARTITION BY. . . .	145
17.2	Window frame specifications on a 7-row result set.	148

18.1	The five SQL join types.	154
18.2	Three tables joined in the F1 self-join example.	155
19.1	Use The Index, Luke!	157
22.1	timestamp vs. timestamptz across session timezones.	186
22.2	Three range type operators on a number line.	199
24.1	The Postgresql object model manager for PHP	217
27.1	The forum MVP schema.	231
28.1	Normalization from un-normalized through 2NF to 3NF.	240
28.2	A normalized address schema for delivery systems.	244
29.1	The normalized Geonames schema.	262
29.2	French GeoNames places coloured by region, over departments and rivers.	263
29.3	GeoNames places across Europe, coloured by feature class (1% sample).	270
30.1	EAV anti-pattern versus normalized design.	273
30.2	Violating 1NF with a CSV column versus a normalized three- table design.	277
35.1	MVCC row visibility under concurrent transactions.	313
36.1	SQL standard isolation levels versus concurrency anomalies.	323
36.2	Lock contention between two concurrent transactions.	324
46.1	Geographic footprint of the #Hiring hashtag.	403
47.1	A geographic point is just a pair: (longitude, latitude).	404
47.2	The pubs of the United Kingdom.	408
47.3	A GiST index groups nearby points into bounding boxes for kNN search.	409
48.1	Distance on Earth follows the great circle, not the straight chord.	411
48.2	The ten nearest pubs to Holborn (order by pos <-> point).	413
50.1	Density of 200,000 geolocated tweets across North America.	429

Index

ACID, 20, 165, 321

Aggregate

array_agg, 205

bool_and, 120, 121, 172

count, 107, 120, 121, 126, 155, 172, 332

count distinct, 432

distinct, 107

filter, 121, 172, 192, 332

median, 194

order by, 205

percentile, 194

sum, 121

within group, 194

Amdahl's law, 81

Anomalies, 241

deletion anomaly, 242

insertion anomaly, 242

update anomaly, 242

anosql, 47

Anti-Patterns, 271

EAV, 271

Multiple Values, 275

Triggers, 340

UUID, 277

array_agg, 205

array_length, 198

array_to_string, 407

Attribute Value, 166

Author

Dimitri Fontaine, xiv

bernouilli, 171, 268

between, 333

bool_and, 120, 121

Cache Invalidation, 332

calendar, 166

case, 16, 110, 119

Cast, 432

cast, 13, 16

catalog, 168

pg_am, 169, 423

pg_amop, 169, 423

pg_catalog, 171

pg_opclass, 169, 423

pg_operator, 168, 182

pg_pgnamespace, 171

pg_type, 171

regoperator, 169

regproc, 168

regtype, 168

ceil, 425

Citus, 297, 441

clock_timestamp, 189

Clojure, 359

coalesce, 13, 16

comments, 66

Common Lisp, 163, 329, 385, 405, 431

consistency, 165

Constraints

Check, 249

Exclusion, 250

Foreign Keys, 248

Not Null, 249

Primary Key, 245

Surrogate Key, 245

Unique, 248

contrib, 368

COPY, 4, 196, 204, 385, 405, 431

count, 104, 107, 120, 121, 126, 155

create table, 153

CTE

recursive, 131

cube, 125

current_setting, 296

Data Domain, 166

Data Set

A Midsummer Night's Dream,
315

Access Log, 196

cdstore, 43

Chinook, 45

commilog, 192

fidb, 91

Geonames, 251

HydroRIVERS, 131

IMF, 200

International Monetary Fund,
200

Last.fm, 383

Lorem Ipsum, 229

Maxmind Geolite, 419

MoMA, 353, 378

Open Street Map, 405

Pub Names, 405

Rates, 200

sandbox, 227

Scan34, 196

The Museum of Modern Art
Collection, 353

Tweets, 204

Data Type, 166, 171

Array, 203

arrays, 398

bigint, 181

bigserial, 183

boolean, 172

Bytea, 185

character, 174

cidr, 196

composite, 209

date, 166

double precision, 181

inet, 196

integer, 181

interval, 167, 190

ip4r, 169

ipaddr, 433

JSON, 24, 211, 298, 385

JSONB, 24, 211, 293

macaddr, 196

Network Address Types, 196

number, 181

numeric, 181

point, 408

query_int, 400

range, 199, 286

real, 181

sequence, 183

serial, 183

smallint, 181

text, 174

Time Zones, 186

timestamp, 166, 187

timestampz, 187

- UUID, 184, 277
- varchar, 174
- XML, 210
- Database Anomalies, 241
- Date
 - allballs, 167
 - clock_timestamp, 189
 - date_trunc, 121
 - day, 102
 - extract, 113
 - generate_series, 191
 - interval, 102
 - isodow, 102, 193
 - isoyear, 102
 - Time Zones, 186
 - to_char, 193
 - week, 102
 - year, 102
- DCL, 95
- DDL, 95
 - alter table, 320, 384
 - alter user set, 295
 - cascade, 232
 - check, 249
 - create database, 226
 - create domain, 249
 - create extension, 368, 376, 391, 398, 412
 - create function, 296, 437
 - create index, 206, 213, 235, 266, 283, 396, 399, 409, 414
 - create materialized view, 283, 334, 399
 - create or replace function, 338, 341
 - create role, 295
 - create schema, 226
 - create schema if not exists, 285
 - create table, 153, 184, 212, 214, 227, 244, 251, 285, 293
 - create table like, 320
 - create trigger, 338, 341, 345, 378
 - create type, 215, 285, 327
 - create unique index, 334
 - create view, 333, 399
 - drop schema, 232
 - drop table, 293
 - drop table if exists, 215
 - exclude using, 250, 286
 - foreign key, 247
 - primary key, 247
 - references, 247
 - refresh materialized view concurrently, 335
 - trigger, 296
 - truncate, 319
 - unique, 248
- default, 184
- desc, 107
- diff, 100, 357, 377
- distance, 266
- distinct, 107
- distinct on, 134, 197
- Django, 84
- DML, 95
 - delete, 317
 - delete returning, 318, 435
 - insert into, 309
 - insert on conflict do update, 349, 435
 - insert returning, 355, 435
 - insert select, 205, 214, 310
 - on conflict do nothing, 357
 - truncate, 319
 - update, 312, 324
 - update returning, 313, 326, 355
- DRY, 223

- encoding, 178
 - client_encoding, 179
 - server encoding, 178
- enum, 215
- except, 136, 394
- Exclusion Constraints, 250
- explain, 109, 110, 112, 207, 267, 397, 410, 415
- extensibility, 203
- Extension, 216, 364
 - contrib, 368
 - cube, 412
 - earthdistance, 411
 - hll, 427
 - hstore, 285, 376
 - hyperloglog, 427
 - intarray, 289, 398
 - ip4r, 419
 - ltree, 289
 - pg_trgm, 289, 367, 390
 - PL/XSLT, 210
- extract, 102, 126
- fetch first rows only, 117
- filter, 121, 172
- format, 101, 400
- from, 97, 103
- generate_series, 13, 16, 102, 227
- Geolocation, 251
- Geonames, 251
- Go, 348
 - listen, 352
 - notify, 352
- group by, 40, 46, 104, 107, 113, 118, 172, 265
 - cube, 125
 - grouping sets, 123, 282, 283, 433, 439
 - rollup, 124, 177
- grouping sets, 123
- having, 40, 122, 172, 198
 - sum, 282
- histogram, 127, 193, 265
- hll_add_agg, 433
- hll_hash_text, 433
- hll_union_agg, 439
- Index
 - B-Tree, 77, 235
 - bloom, 77
 - BRIN, 77
 - GIN, 77, 213, 399
 - gin, 206
 - gin_int_ops, 399
 - GiST, 77, 266, 396, 409
 - gist, 169
 - gist_trgm_ops, 396
 - Hash, 77
 - jsonb_path_ops, 213
 - SP-GiST, 77
- interval, 102
- Interview
 - Alvaro Hernandez Tortosa, 298
 - Craig Kerstiens, 441
 - Gregoire Hubert, 218
 - Kris Jenkins, 359
 - Markus Winand, 157
 - Yohann Gabory, 84
- is false, 172
- is null, 172
- is true, 172
- Isolation, 321
 - Dirty Read, 322
 - Non-Repeatable Read, 322
 - Phantom Read, 322
 - Serializable, 324

Serialization, 322
 SSI, 324
 Isolation Levels, 36
 Java, 98, 100
 listen, 352
 notify, 352
 join, 40, 104, 107, 113, 126, 156, 265, 381, 387, 425
 cross join, 139, 143
 full outer, 156
 inner, 156
 insert, 214
 lateral, 113, 156, 293, 414
 lateral join, 207
 left, 156
 left join, 16, 46, 68, 106, 142, 155, 227, 261, 266, 316
 left join lateral, 113, 208, 234, 266, 283, 425
 on true, 208, 266, 283, 425
 outer, 156
 outer join, 142
 subquery, 106, 208, 266, 283, 414, 425
 using, 266, 316
 JSON, 24, 211, 385
 json_each, 349
 json_populate_record, 349
 JSONB, 24, 211
 jsonb_array_elements, 293
 jsonb_each, 293
 jsonb_pretty, 24, 234
 kNN, 111, 137, 408
 lag, 16, 150
 lateral, 40
 lc_time, 195
 lead, 150

leap year, 103
 left join, 13
 limit, 104, 109, 259
 Lisp, 329
 Listen, 344
 Little Bobby Tables, 9
 lock table, 357
 Lorem Ipsum, 229
 Modelisation
 Anti-Patterns, 271
 Audit Trails, 284
 Check Constraints, 249
 Database Anomalies, 241
 Denormalization, 279
 Exclusion Constraints, 250
 Foreign Keys, 227, 248
 History Tables, 284
 Indexing, 235
 JSON, 292
 Lorem Ipsum, 229
 Materialized Views, 282
 Normal Forms, 240
 Normalization, 237
 Not Null Constraints, 249
 Not Only SQL, 291
 Partitioning, 288
 Primary Keys, 227, 245
 Schemaless, 292
 Surrogate Keys, 245
 Music
 AC/DC, 64
 Aerosmith, 387
 Black Sabbath, 66
 Iron Maiden, 66
 Maceo Parker, 395
 Red Hot Chili Peppers, 32
 MVCC, 74
 MVP, 225

- no offset, 116
- Normal Forms, 240
 - 1NF, 240, 275, 306
 - 2NF, 240, 256, 306
 - 3NF, 240, 306
 - 4NF, 240
 - 5NF, 240
 - BCNF, 240
 - DKNF, 240
- NoSQL, 24, 291
- not exists, 107
- not found, 341
- not null, 141
- Not Only SQL, 291
- Notify, 344
- now, 188
- ntile, 150
- null, 139, 142
- offset, 116, 259
- Open Street Map, 405
- Operators
 - >, 380
 - ::, 432
 - >, 412
 - <->, 266, 392, 408, 412
 - <>, 355
 - »=, 424
 - >, 24, 207, 208, 212
 - , 392
 - *, 384, 392
 - between, 333
- order by, 13, 16, 46, 104, 107, 109–III, 113, 131, 172, 265
 - is not null, 282
 - nulls first, 433
 - nulls last, 148
 - order by case, 110
 - order by distance, III, 137, 408, 412, 414
 - order by sum, 121
 - window function, 146
- over, 16, 146, 150
- partition by, 16
- Partitioning, 288
- People
 - Alan Kay, 222
 - Alvaro Hernandez Tortosa, 298
 - Amdahl, 81
 - Andrew Gierth, 231, 419
 - Anton Chekhov, 445
 - Craig Kerstiens, 441
 - Dimitri Fontaine, xiv
 - Donald Knuth, 276, 326
 - Edsger Wybe Dijkstra, xiii
 - Fred Brooks, 222
 - Gregoire Hubert, 218
 - Julien Danjou, xiv
 - Kris Jenkins, 359
 - Lawrence A. Rowe, 364
 - Linus Torvalds, 2
 - Markus Winand, 116, 157
 - Martin Fowler, 332
 - Michael Stonebraker, 203, 364
 - Phil Karlton, 332
 - Rob Pike, xii, 238, 290
 - Shakespeare, 315
 - Tom Lane, 3
 - Yohann Gabory, 84
- percentile_cont, 194
- pg_column_size, 187
- pg_database_size, 61
- pg_stat_statements, 441
- pg_typeof, 168
- pgloader, 44, 91, 253, 383, 420
- PHP, 218

- PLpgSQL, 41
- populate_record, 381
- PostGIS, 417
- Programming Language
 - Common Lisp, 163, 329, 385, 405, 431
 - Go, 348
 - Java, 98, 100
 - Lisp, 329
 - PHP, 218
 - Python, 7, 33, 50, 293
- psql, 45, 55
 - columns, 174, 267
 - ECHO_HIDDEN, 60
 - EDITOR, 56
 - format, 174, 267
 - include, 228
 - intervalstyle, 56
 - LESS, 56
 - ON_ERROR_ROLLBACK, 56, 326
 - ON_ERROR_STOP, 56
 - PROMPTI, 56
 - pset, 56
 - psqlrc, 56
 - REPL, 225
 - set, 56, 282
 - setenv, 56
- psqlrc, 60
- Python, 7, 33, 50, 293
 - anosql, 47
 - listen, 352
 - notify, 352
- Queries, 95
- query_int, 400
- random, 233
- RDBMS, 20, 165
- recursive query, 131
- references, 227
- regex, 175
- regexp_matches, 205
- regexp_split_to_array, 176
- regexp_split_to_table, 175, 176
- regresql, 70
- Regular Expression, 175
- relation, 152, 165
- relational, 165
- Relational algebra, 153
- REPL, 225
- replace, 407
- rollup, 124
- round, 16, 266, 412
- row_number, 150
- rows between, 146
- sample, 171
- sampling, 268
- Scale Out, 296
- Schemaless, 292
- search_path, 226
- select, 97
- select star, 98
- self join, 128
- server_version, 26
- set local, 296
- set_masken, 197
- setval, 184
- share row exclusive, 357
- show_trgm, 391
- similarity, 391
- SQL, 27–443
- SRF, 207, 293
- Stored Procedure, 41, 437
- subquery, 64, 106
- subselect, 414
- substring, 194, 208

- sum, 121
- Surrogate Keys, 245
- synchronous_commit, 296
- table, 174, 209, 423
- table of truth, 139
- tablesample, 171, 268
- Tahiti, 188
- TCL, 95
 - begin, 438
 - commit, 58
 - isolation level, 326
 - repeatable read, 326
 - rollback, 58, 341, 435, 438
 - start transaction, 326
- three-valued logic, 139
- timezone, 188
- to_char, 5, 13, 16, 439
- TOAST, 101
- top-N, 113, 234
- ToroDB, 298
- transaction, 165
- Transaction Isolation, 321
- Trigger, 338, 341, 345, 378
- triggers, 337
- Trigrams, 390
- Tuple, 166
- union, 135
- union all, 132, 135
- unique violation, 341
- Unix
 - Basics of the Unix Philosophy, 238
 - Notes on Programming in C, 238
- unnest, 207
- using, 68
- UUID, 277
- uuid_generate_v4, 184
- values, 215
- where, 97, 106
- wikipedia, 128
- window function, 16, 145
 - array_agg, 146
 - lag, 16, 150
 - lead, 150
 - ntile, 150
 - order by, 16, 146, 148
 - over, 16
 - partition by, 16, 148
 - row_number, 148, 150
 - sum, 147
- winners, 128
- with, 16, 40, 68, 113, 119, 120, 126, 128, 177, 205, 415, 435
 - delete insert, 435
 - delete returning, 435
 - insert returning, 355
 - update returning, 355
- with delete, 318
- with recursive, 131
- within group, 194
- XKCD, 9
- XML, 315, 405
- YeSQL, 24, 359

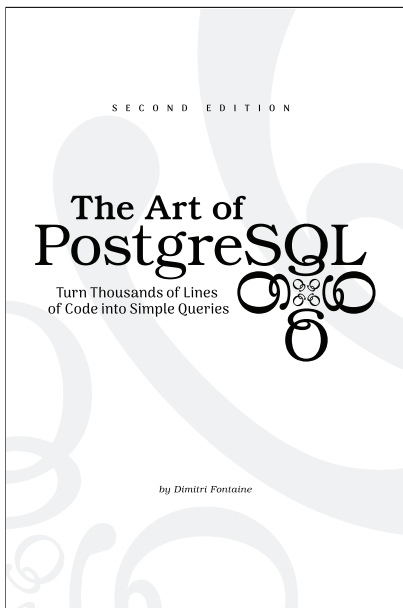
I have written THE ART OF POSTGRESQL so that as a developer, you may think of SQL as a full-blown programming language. Some of the problems that we have to solve as developers are best addressed using SQL.

The Art of PostgreSQL

Turn Thousands of Lines
of Code into Simple Queries



Did you buy the book already?



This was the *sample* of the book THE ART OF POSTGRESQL, with the full table of contents, the introduction part of the book, and the full list of figures and index. If you did like what you've read and want to improve you SQL skills, consider buying the book now!

Buy the book online at TheArtOfPostgreSQL.com now!